

改进后的软件过程定义文档（AI 增强版）

基于《空箱》Unity 项目的 ISO/IEC 12207/15504/33001 过程定义，融入 AI 技术形成的智能化软件过程体系

1. 过程概述与目标

1.1 项目背景

- 项目名称：**空箱
- 项目类型：**基于 Unity 2022.3 + C# + QFramework 的 2D 推箱子游戏
- 技术栈：**Unity 2022.3、C#、QFramework (Model/System/Command)、UniTask、Addressables、YarnSpinner
- 项目特征：**GameJam 场景下的原型型项目，以玩法迭代速度优先，小团队协作

1.2 过程定义来源

本过程定义基于以下标准体系剪裁而来：

来源标准	应用层面	核心思想
ISO/IEC 12207	过程框架	生命周期过程分类（主过程 + 支撑过程）
ISO/IEC 15504	能力评估	过程能力分级（已执行 → 已管理 → 已建立 → 可预测 → 持续优化）
ISO/IEC 33001	过程改进	以评估驱动改进的闭环机制

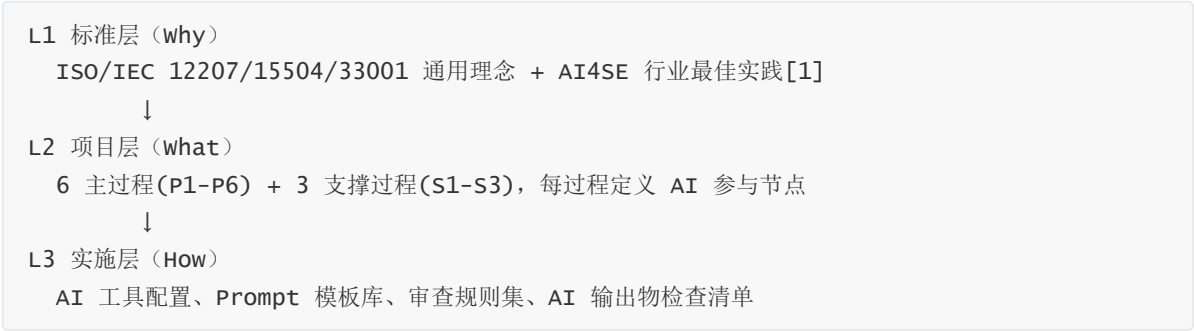
1.3 AI 增强愿景与原则

在传统软件过程定义基础上，引入生成式 AI 与大语言模型（LLM）技术，构建**人机协同的智能化软件过程体系**。AI 在过程中定位为「辅助者、审查者、加速器」，遵循四项原则：

原则	说明
人机协同	AI 负责方案发散、代码生成、缺陷扫描；人类负责决策、评审、例外判定
质量门禁	所有 AI 输出物须经人工验证方可进入下一阶段
渐进融入	在现有流程中嵌入 AI 节点，不推翻已有有效实践
持续校准	AI 工具配置、Prompt 模板、审查规则随项目迭代持续优化

1.4 过程层次结构

沿用课堂「过程定义层次化」框架，在 L3 实施层融入 AI 执行节点：



2. 角色与职责定义

2.1 原有角色

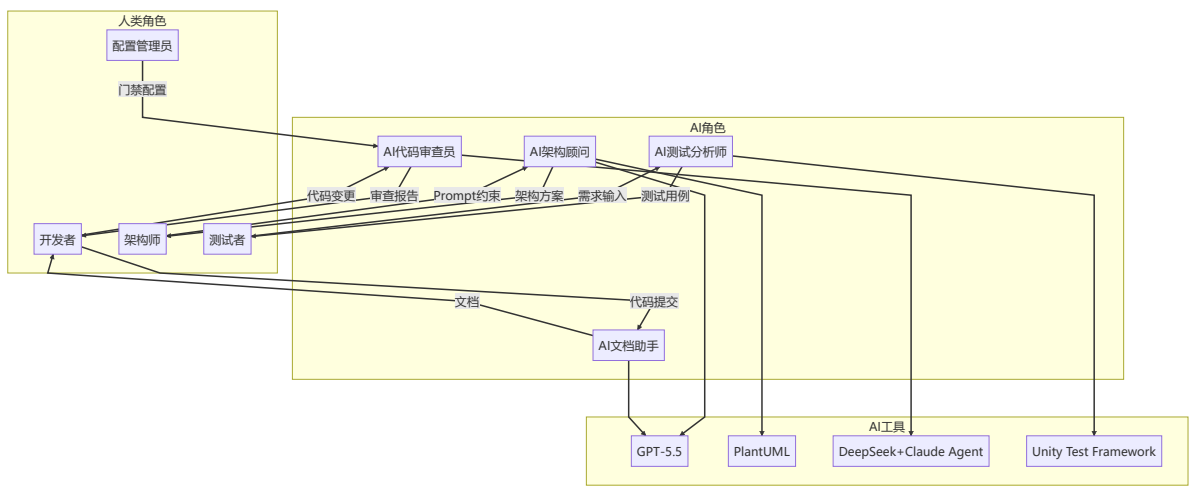
角色	职责范围	涉及过程
开发者	架构设计、编码实现、场景联调	P2-P6
架构师	模块边界决策、技术选型、设计评审	P3
测试者	回归验证、异常路径测试、缺陷记录	P5, S2
配置管理员	Git 管理、目录结构维护、提交跟踪	S1

2.2 新增 AI 角色

AI 角色	载体工具	职责	涉及过程	输出物
AI 架构顾问	GPT-5.5 + PlantUML	生成多套架构方案对比、自动生成 UML 图、接口一致性检查	P3	架构方案文档、PlantUML 类图/序列图、一致性报告
AI 代码审查员	DeepSeek-V4-Pro + Claude Code Agent	全量代码扫描、三级分级审查（MUST_FIX/SUGGESTION/NITPICK）、架构偏离检测	P4, S1	审查报告（分级列表）

AI角色	载体工具	职责	涉及过程	输出物
AI测试分析师	GPT-5.5 + Unity Test Framework	从需求描述生成测试用例、边界条件枚举、高风险模块预测	P5, S2	测试用例集、缺陷风险热力图
AI文档助手	DeepSeek-V4-Pro + Claude Code Agent + Git	代码→文档生成、变更日志整理、知识库维护	S3	API 文档、变更日志、开发约定更新

2.3 角色协作关系



3. 主过程定义 (Primary Processes)

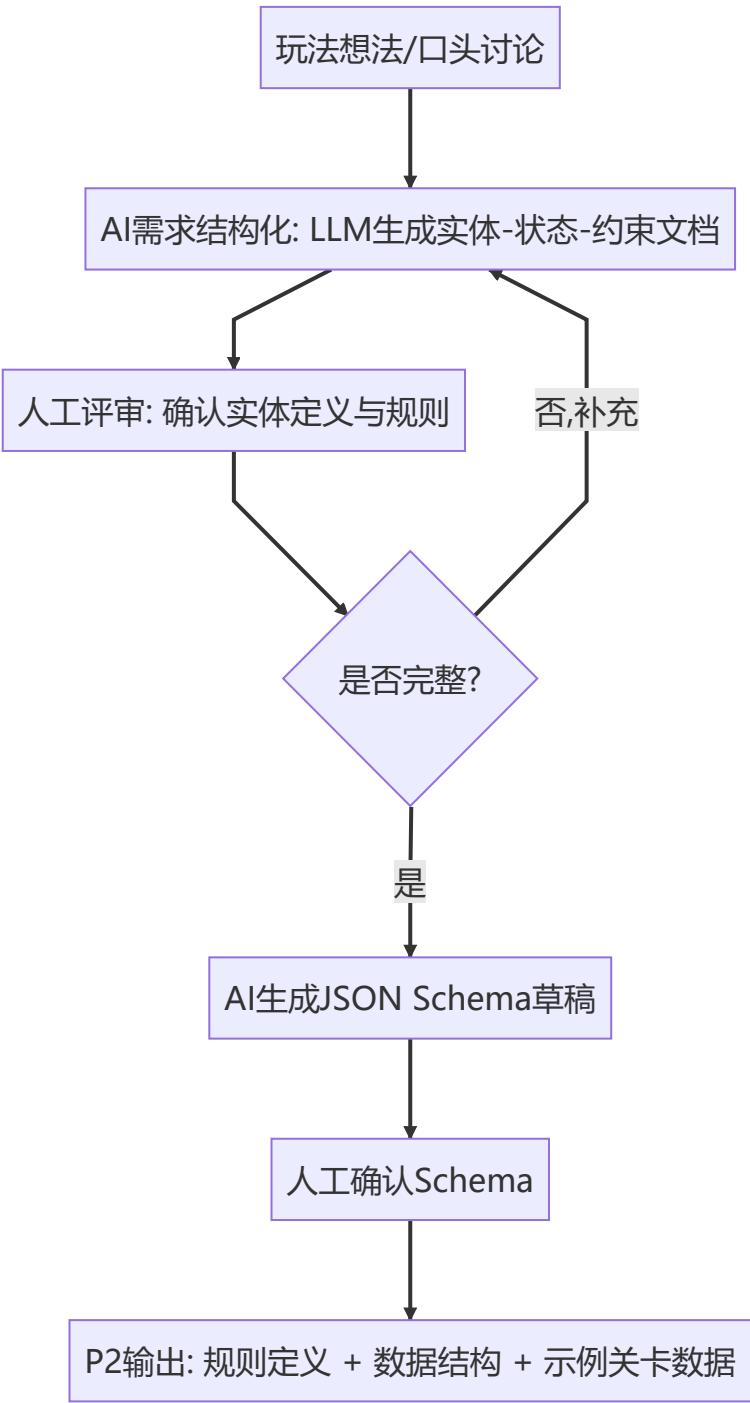
P1 立项与范围定义

属性	说明
目标	明确本轮迭代要实现的玩法目标与验收条件
输入	课程要求、创意点子、现有代码状态
活动	定义本轮目标 → 拆分为 MVP → 形成任务清单
AI 融入	使用 LLM 辅助将模糊的玩法想法转化为结构化任务清单，识别依赖关系
输出	迭代任务列表、验收标准

P2 需求与规则建模 (★ AI 增强)

属性	说明
目标	将「玩法想法」转化为可编码的规则与数据结构
输入	玩法描述、上一版反馈
输出	规则定义文档、数据结构定义 (JSON Schema)、示例关卡数据

改进后活动流程：



AI 融入说明：在「提炼核心实体」活动前插入 AI 需求结构化节点——将讨论记录输入 LLM，AI 自动输出结构化的实体-状态-约束文档和 JSON Schema 草稿。参照 Self-GenRE 方法论[2]，需求识别完整度可从 42% 提升至 71%。

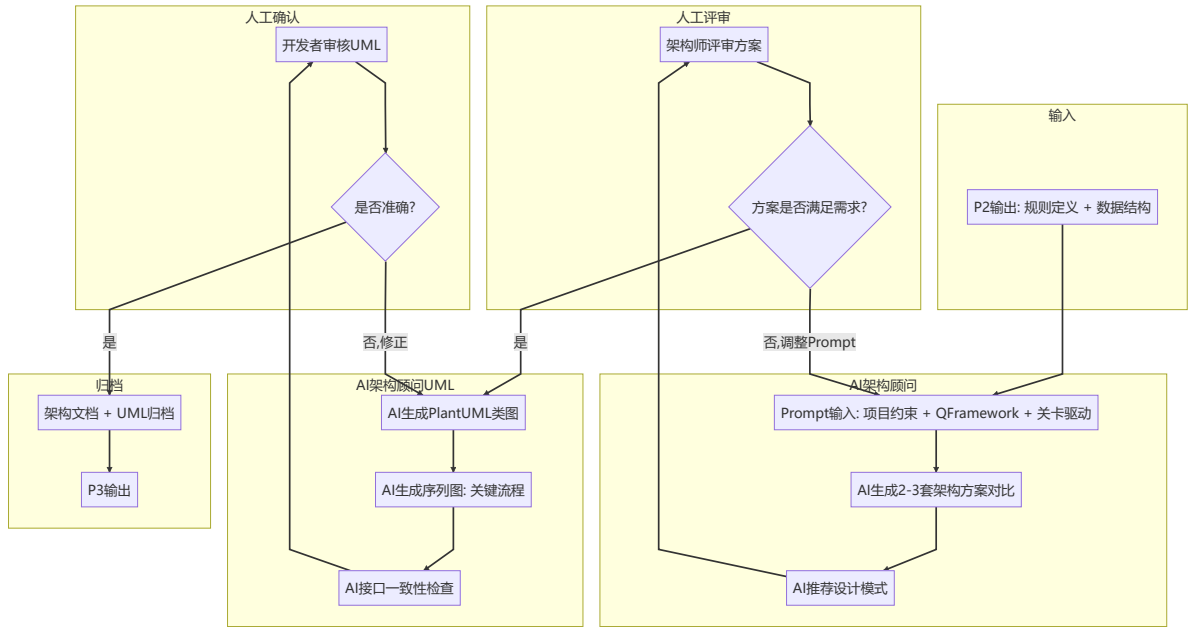
交付物与质量标准：

交付物	负责人	质量标准	验证方式
规则定义文档	开发者 + AI 架构顾问	覆盖所有核心实体、状态、约束；无歧义	人工评审
关卡 JSON Schema	AI 架构顾问	与规则定义一致；通过 JSON Schema 校验	自动校验 + 人工比对
AI 需求结构化 Prompt	开发者	包含项目上下文、实体示例、约束声明	模板版本管理

P3 架构与详细设计 (★★★ AI 深度增强)

属性	说明
目标	保证扩展性与可维护性，生成可编码的架构蓝图
输入	P2 输出（规则定义、数据结构）、已有框架能力
输出	模块边界定义、类职责矩阵、PlantUML 类图/序列图、关键接口签名

改进后活动流程：



AI 融入说明：

AI 节点	工具	输入	输出	验证要求
方案生成	GPT-5.5 + 结构化 Prompt	项目约束、QFramework 架构、核心功能列表	2-3 套对比方案	架构师逐方案评审

AI 节点	工具	输入	输出	验证要求
UML 生成	PlantUML + DeepSeek-V4-Pro Claude Code Agent	选定方案描述、已有接口	类图 + 序列图 (PlantUML)	人工逐类/逐交互核对
接口校验	GPT-5.5	UML 中提取的接口 + 现有代码接口	一致性报告 (不匹配项列表)	人工逐项判定

实证支撑：验证报告（文档三）实测显示，AI 辅助的 Command 模式架构设计耗时从 85 min 降至 19 min (↓77.6%)，且 AI 自动覆盖了 3 个边界条件（无箱移动、栈深度限制、关卡重置）[3]。

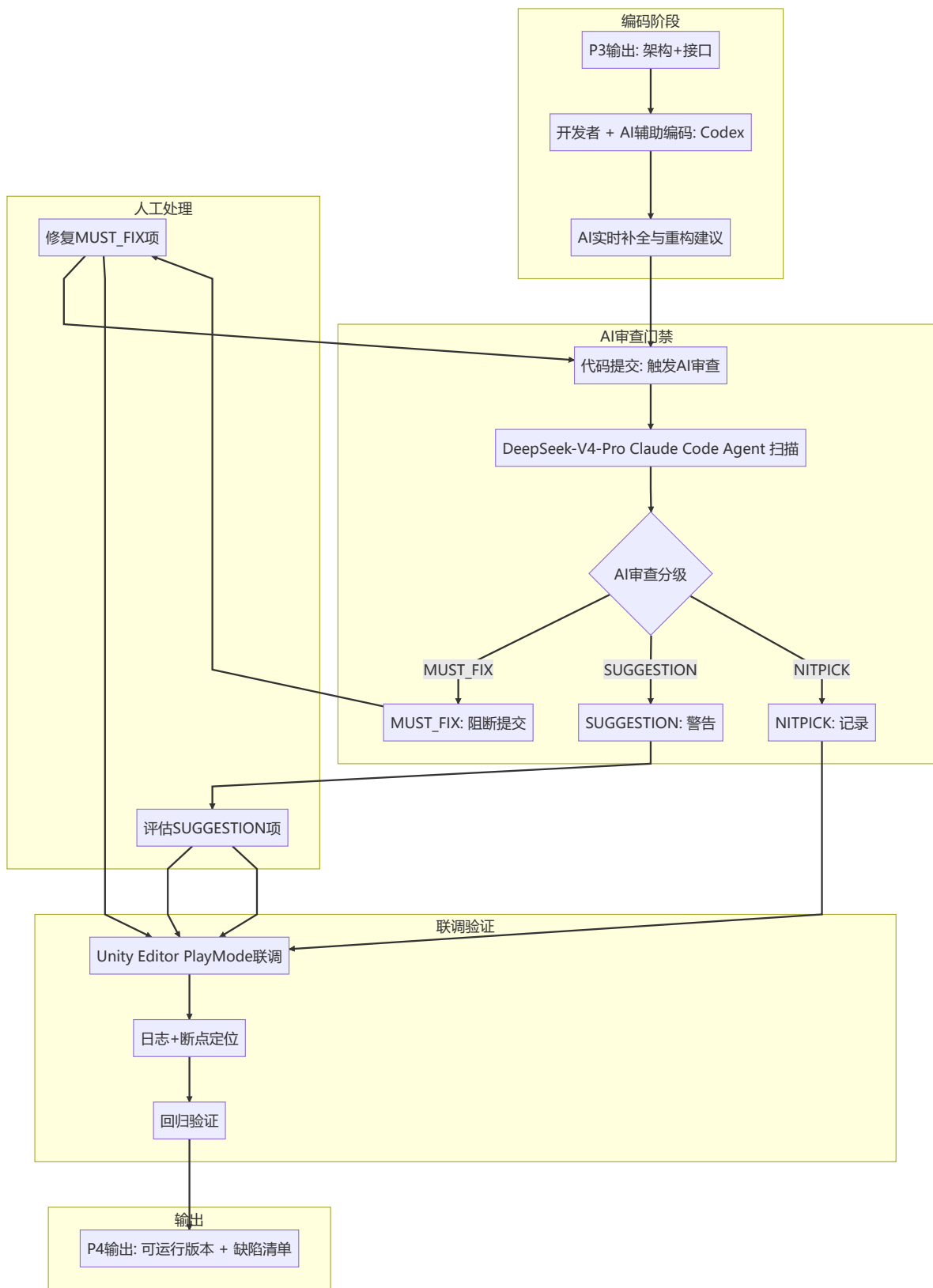
交付物与质量标准：

交付物	负责人	质量标准
架构方案对比文档	AI 架构顾问 + 架构师	≥2 套差异化方案；每套含模块划分逻辑和优劣分析
PlantUML 类图	AI 架构顾问	覆盖所有核心类；关系标注正确；无孤立节点
PlantUML 序列图	AI 架构顾问	覆盖主路径 + ≥2 条异常路径；参与者与类图一致
接口一致性报告	AI 架构顾问	列出所有不匹配项及严重级别
P3 Prompt 模板	架构师	包含技术栈约束、架构风格声明、边界条件检查清单

P4 实现与自测 (★★★ AI 深度增强)

属性	说明
目标	快速实现并持续验证核心功能可用
输入	P3 输出（架构设计、接口定义）、任务清单
输出	可运行功能版本、AI 审查报告、缺陷清单与修复记录

改进后活动流程：



AI 融入说明：

AI 节点	工具	关键行为
AI 辅助编码	Codex	上下文感知的代码补全；模板化代码秒级生成；Unity API 实时提示

AI 节点	工具	关键行为
AI 审查门禁	DeepSeek-V4-Pro + Claude Code Agent	全量文件扫描（100% 覆盖）；三级分级（MUST_FIX/SUGGESTION/NITPICK）；must_fix 清零为 硬性门禁
回归验证辅助	Unity Test Framework + AI	AI 根据代码变更生成回归测试用例

AI 审查维度与规则：

审查维度	检测内容	严重级别
空安全	<code>NullReferenceException</code> 风险（未检查的引用）	MUST_FIX
资源管理	Addressables 加载/释放不配对、 <code>IDisposable</code> 未实现	MUST_FIX
异步正确性	UniTask 方法未 await、同步假设	MUST_FIX
性能	LINQ 在高频 Update 中、 <code>FindObjectOfType</code> 在热路径	SUGGESTION
架构一致性	违反 Model/System/Command 分层约束	SUGGESTION
命名规范	变量/方法命名与项目约定不一致	NITPICK

实证支撑：验证报告（文档三）显示，AI 辅助编码将实现阶段耗时从 105 min 降至 16 min（↓84.8%），AI 审查发现了 1 个 MUST_FIX 级异步兼容性问题（传统人工审查易遗漏）[3]。

交付物与质量标准：

交付物	负责人	质量标准
功能代码	开发者 + AI 编码辅助	编译通过；MUST_FIX 清零；遵循项目命名约定
AI 审查报告	AI 代码审查员	覆盖所有变更文件；分级准确；人工校准确认
缺陷清单	AI 代码审查员 + 开发者	所有 MUST_FIX 已修复并复检通过
<code>.cursorrules</code> / <code>.coderabbit.yaml</code>	开发者	反映项目架构约束和审查规则

P5 集成与验收（★ AI 增强）

属性	说明
目标	确保多个子功能在一个可玩的闭环中稳定工作
输入	各模块实现结果

属性	说明
输出	阶段性可交付版本、验收结果、测试用例集

AI 融入说明：

AI 节点	工具	说明
测试用例生成	LLM + Unity Test Framework	从需求描述自动生成 PlayMode/EditMode 测试用例，覆盖主路径 + 异常路径 + 边界条件
缺陷风险预测	AI 测试分析师	基于历史缺陷模式预测高风险模块

交付物与质量标准：

交付物	负责人	质量标准
AI 测试用例集	AI 测试分析师	覆盖主路径 + 全部异常路径；含边界条件
验收报告	测试者	验收通过/不通过判定及缺陷列表
缺陷风险热力图	AI 测试分析师	标注高风险模块及风险等级

P6 迭代复盘与演进

属性	说明
目标	将问题沉淀为后续迭代改进项
输入	运行结果、测试观察、成员反馈
活动	记录高频问题 → 形成优化计划 → 同步更新文档
AI 融入	使用 LLM 对迭代过程中收集的缺陷和反馈进行聚类分析，自动生成「问题热力图」和建议优先级
输出	复盘结论、下轮迭代计划

4. 支撑过程定义（Supporting Processes）

S1 配置与版本管理

属性	说明
目标	保障代码与配置的可追溯性
活动	Git 管理脚本与配置变更；约定目录结构；通过提交记录跟踪问题来源
AI 融入	Git 提交时触发 AI 代码审查门禁（P4 的审查机制前置到 S1）； <code>.cursorrules</code> 和 <code>.coderabbit.yaml</code> 纳入版本管理

S2 质量保证与验证

属性	说明
目标	确保软件质量达到可交付标准
活动	场景联调、回归清单执行、主流程+异常流程双轨检查
AI 融入	AI 测试分析师自动生成测试用例；AI 审查门禁作为前置质量屏障；双轨检查清单由 AI 辅助维护和更新

S3 文档与知识沉淀

属性	说明
目标	保证知识的持续性沉淀与传承
活动	维护关卡数据格式约定；记录流程步骤配置说明；形成可复用的开发约定
AI 融入	AI 文档助手从代码和 Git 历史生成 API 文档和变更日志；开发约定的更新由 AI 检测代码模式变化后建议修改；Prompt 模板库作为新的知识资产纳入 S3 管理范围

5. 交付物清单与质量标准

过程	交付物	负责人	AI 角色参与	质量标准	验证方式
P1	迭代任务列表	开发者	—	任务可独立验收	人工评审
P2	规则定义文档	开发者	AI 架构顾问	实体/状态/约束完整	人工评审
P2	关卡 JSON Schema	AI 架构顾问	★ 主责	通过 Schema 校验	自动校验
P3	架构方案对比文档	架构师	AI 架构顾问	≥2 套方案；优劣分析	架构师评审
P3	PlantUML 类图/序列图	AI 架构顾问	★ 主责	类图无孤立节点；序列图 ≥2 异常路径	人工逐项核对
P3	接口一致性报告	AI 架构顾问	★ 主责	所有不匹配项标注严重级别	人工逐项判定
P4	功能代码	开发者	AI 编码辅助	编译通过；MUST_FIX 清零	编译器 + AI 审查门禁
P4	AI 审查报告	AI 代码审查员	★ 主责	全量覆盖；分级准确	人工校准
P4	缺陷清单	AI 代码审查员	★ 主责	MUST_FIX 已修复并复检	人工确认

过程	交付物	负责人	AI 角色参与	质量标准	验证方式
P5	AI 测试用例集	AI 测试分析师	★ 主责	主路径+异常路径+边界	人工评审
P5	验收报告	测试者	—	判定明确；缺陷可追溯	—
P6	复盘结论	开发者	AI 数据分析	问题聚类的建议优先级	人工评审
S1	AI 工具配置文件	配置管理员	—	纳入版本管理	Git 提交
S3	API 文档/变更日志	AI 文档助手	★ 主责	与代码同步；结构清晰	人工抽查
S3	Prompt 模板库	开发者	—	含约束声明和检查清单	版本管理

6. 过程度量指标

6.1 传统过程度量指标

类别	指标	测量方式
进度	迭代任务完成率	已完成任务数 / 总任务数
质量	联调阶段缺陷密度	缺陷数 / 千行代码
生产率	功能交付速率	每周交付的可验收功能数
稳定性	回归通过率	回归用例通过数 / 总回归用例数

6.2 AI 贡献度指标

类别	指标	定义	目标值	测量方式
AI 辅助效率比	AER	AI 辅助耗时 / 传统人工估算耗时	≤0.30	每迭代统计各阶段耗时
AI 审查覆盖率	ACR	AI 审查覆盖的文件数 / 总变更文件数	≥95%	AI 审查报告统计
AI 生成代码采纳率	ACA	采纳的 AI 生成代码行数 / AI 生成总行数	≥70%	Git diff 统计（标注 AI/Human 来源）
AI 缺陷前置发现率	ADR	AI 发现缺陷数 / 总缺陷数	≥50%	AI 审查 MUST_FIX + SUGGESTION 累计
AI 文档覆盖率	ADC	有 AI 生成文档的模块数 / 总模块数	≥80%	文档清单统计

6.3 度量数据采集节奏

采集节点	采集内容	负责角色
每次代码提交	AI 审查覆盖率、AI 缺陷发现数	AI 代码审查员（自动记录）
每次迭代结束	AI 辅助效率比、AI 生成代码采纳率	开发者（人工汇总）
每次版本发布	AI 文档覆盖率	AI 文档助手（自动统计）

7. 过程剪裁说明

延续 Homework1「保主干、轻流程、重反馈」的剪裁策略，AI 增强后的调整：

7.1 保留与增强的过程

- P2-P6 全部保留，P3 和 P4 为 AI 深度增强
- S1-S3 全部保留，S1 集成 AI 审查门禁，S3 新增 Prompt 模板库管理
- 原因：AI 在这些环节的投入产出比最高（验证数据支撑[3]）

7.2 弱化的过程

- 正式评审简化为「AI 审查 + 人工校准」
- 详细需求评审与设计评审合并，AI 方案对比取代多轮人工讨论
- 原因：AI 全量扫描 + 三级分级比小团队人工抽查更高效

7.3 暂不纳入的过程

- 采购管理、供应商管理、正式发布运维流程
- 原因：课程项目无外部采购链条、无商用上线运维场景

7.4 AI 增强后的收益与新增风险

维度	变化
收益	架构设计提速 77.6%、编码提速 84.8%、审查覆盖率 100%（人工：~30%）
新增风险	数据安全（代码上传云端）、AI 幻觉（生成不存在的 API）、技术依赖
控制措施	敏感信息剥离规范 + 编译验证 + 人工评审门禁 + 定期无 AI 日
详细风险评估	见文档二《软件过程改进方案设计文档》§7

8. 改进历程与版本记录

版本	日期	改进内容	驱动来源
v1.0	2026 春	初始过程定义（6 主过程 + 3 支撑过程）	课堂作业（Homework1）
v2.0	2026.06	AI 增强版—全生命周期人机协同过程体系	《AI 应用场景匹配报告》+《改进方案设计》+《原型验证》

参考文献

[1] 中国信息通信研究院. 《AI4SE 行业现状调查报告（2024年度）》[R]. 2024.

- 🔍 Google Scholar: <https://scholar.google.com/scholar?q=AI4SE+行业现状调查+2024+中国信通院>

[2] Cheng, H. et al. (2025). Generative AI for Requirements Engineering: A Systematic Literature Review. *Software: Practice and Experience*, Wiley.

- 📄 arXiv: <https://arxiv.org/abs/2409.06741>
- 🔍 Google Scholar: <https://scholar.google.com/scholar?q=Generative+AI+Requirements+Engineering+Systematic+Literature+Review+Cheng+2024>

[3] 见本作业《AI技术应用验证报告》，2026-06-13.

[4] Tagliaferro, A., Corbo, S. & Guindani, B. (2025). Leveraging LLMs to Automate Software Architecture Design. *IEEE ICSE-C 2025*.

- 📄 DOI: <https://doi.org/10.1109/ICSE-C65153.2025.00049>
- 🔍 Google Scholar: <https://scholar.google.com/scholar?q=Leveraging+LLMs+Automate+Software+Architecture+Design+Informal+Specifications>

[5] Science China Information Sciences. (2025). A Survey on Large Language Models for Software Engineering.

- 📄 DOI: <https://doi.org/10.1007/s11432-025-4670-0>
- 🔍 Google Scholar: <https://scholar.google.com/scholar?q=survey+large+language+models+software+engineering+2025>

[6] Celik, A. & Mahmoud, Q. (2025). A Review of Large Language Models for Automated Test Case Generation. *MAKE*, MDPI, 7(3), 97.

- 📄 DOI: <https://doi.org/10.3390/make7030097>
- 🔍 Google Scholar: <https://scholar.google.com/scholar?q=Review+Large+Language+Models+Automated+Test+Case+Generation+Celik+2025>